



CS4135 - Software Architectures Project

Team members

Caylum Hurley - 22356363

Vivian Jently - 22079017

Oscar Zhou -22338705

Joe Considine - 22344977

Darragh Quinn - 22359621

Daragh Downes - 22351159

Department of Computer Science and Information Systems Faculty of Science and
Engineering
University of Limerick

Introduction of the Project

Contribution

Strategic Architecture Development

Part A: Design of architecture based on Model Driven Design

1. Ownership & Division of Bounded Contexts

1.1 Bounded Context Map

1.3 Cross-Context Dependencies

2. DDD Modelling within Each Context

2.1 Identity & Access Context

2.2 Catalogue Context

2.3 Lending Context

2.4 Notification Context

2.5 Book Club Context

3. Aggregate Design & Invariants

3.1 Aggregate Boundary Justifications

3.2 Invariant Summary

4. Inter-Context Contracts

4.1 REST API Contracts (Synchronous)

4.2 Domain Event Contracts (Asynchronous)

4.3 Anti-Corruption Layer Design

5. Conformance to UML/DDD Notation

5.1 Ubiquitous Language Glossary

5.2 UML models

6. API Gateway

6.1 Gateway Responsibilities

6.2 Request Lifecycle

6.3 Relationship to the Cross-Context Dependency Map

Part B: Event Storming & Domain Storytelling

1. Event Storming Outcomes

1.1 Notation Key Used on the Board

1.2 Big-Picture Event Flow

1.3 Hotspots & Conflicts Register

1.4 Policies Discovered

2. Translation of Insights

2.1 Model Changes Introduced by Event Storming

2.2 Backlog Items Generated

3. Domain Storytelling

3.1 DS-01: Student Borrows a Book

3.2 DS-02: Admin Manages Book Inventory

3.3 DS-03: Overdue Book - Reminder and Return Flow

3.4 DS-04: Book Club - Create, Join, and Discuss

4. Facilitation & Evidence

4.1 Session Summary

4.2 Participant Roles in Sessions

4.3 Miro Board Structure & Evidence

4.4 Decisions Log

Part C: Implementation Architecture

1. Build / Run Instructions

2. Services

2.1 Gateway Service

2.2 Catalogue Service

2.3 Lending Service

2.4 Identity Service

2.5 Notification Service

2.6 Book Club Service

3 Configuration Tools

4. Environment Separation

5. Versioning

6. Patterns

7. Integration Tests

8. Traces & Logs

9. Screenshots

Introduction of the Project

Our E-Library system is designed for university staff and students to easily access library inventory. Our system features the ability to manage book inventory, borrow, user access, notifications and book clubs. Our architectural goal was to deploy independent bounded contexts to keep faults isolated and utilize cross context contracts. The bounded contexts can be split into 5:

Identity: authentication, JWT, roles

Catalogue: Book inventory, availability

Lending: Borrowing and returning

Notification: Alerts for overdue books and events

Book club: club creation, membership, messaging

The architecture of our E-library system is built on microservices, where each bounded context is a separate Spring Boot service with a layered architecture within each service going from Domain -> Application -> Infrastructure. To allow for isolation between services and to keep inter-service coupling minimal we use event driven communication.

Contribution

Team Member	Contribution
Joe Considine	Identity & Access context full DDD model (UserAccount aggregate, value objects, domain services), Identity invariants and authentication domain logic. Identity implementation.
Daragh Downes	Bounded context ownership map, Lending context full DDD model (Loan aggregate, domain services), Lending invariants and loan lifecycle design. Lending Implementation.
Caylum Hurley	Context map diagram and cross-context dependency table, API Gateway design and implementation, Book Club event contracts, Ubiquitous language glossary and UML modelling. Miro Board Design.
Darragh Quinn	Book Club context DDD model (BookClub, ClubMembership, Message aggregates), Book Club event contracts. Book club implementation.

Vivian Jently	Notification context DDD model, event contract specifications (LoanCreated, LoanOverdue, MemberJoined). Notification Implementation.
Oscar Zhou	Catalogue context DDD model (Book, BookCopy, Author, Category), Aggregate boundary justifications, ISBN and AvailabilityStatus invariants. Catalogue Implementation.

Strategic Architecture Development

The architecture was built in stages using Domain-Driven Design.

Firstly, we event stormed and identified all the different domain events as well as the commands, actors, and policies that would trigger them. After that, ambiguous decision like race conditions and ownership was handled to ensure everything belonged in the correct context. We then grouped the events into bounded contexts to assign to team members. Working on context mapping was the next priority. We worked on cross-context relationships and discussed things like anti-corruption layers to make integration contracts explicit. Each model was modelled with full DDD constructs like aggregate roots, entities, value objects, repositories, domain services, and domain events with invariants clearly defined. After the planning stage we worked on implementation where each bounded context was implemented as an isolated Spring Boot microservice.

Part A: Design of architecture based on Model Driven Design

1. Ownership & Division of Bounded Contexts

To ensure that each domain has clear ownership and responsibility, the decision was made to decompose our E-Library system into five bounded contexts,

1.1 Bounded Context Map

The five bounded contexts and their ownership:

Bounded Context	Owner	Responsibility	Primary Entities
Identity & Access	Joe Considine	Authentication, authorisation, user profiles, role management	User, Role, JWT Token
Catalogue	Oscar Zhou	Book inventory, search, availability, CRUD operations	Book, Author, Category, Copy
Lending	Daragh Downes	Borrow/return workflows, due dates, loan lifecycle	Loan, LoanStatus, BorrowRequest
Notification	Vivian Jently	Due-date reminders, overdue alerts, confirmations	Notification, NotificationEvent
Book Club	Darragh Quinn, Caylum Hurley	Club creation, memberships, chat/messaging within clubs	BookClub, ClubMembership, Message
API Gateway	Caylum Hurley	Central entry point for all client requests; handles authentication, request routing, and security enforcement before forwarding to bounded contexts	JWT filter, Routing layer, Security context, Gateway request

1.3 Cross-Context Dependencies

The table below shows every cross-context dependency, type of relationship, and the anti-corruption strategy that was used.

Upstream Context	Downstream Context	Relationship Type	Dependency Description
Identity & Access	Catalogue	Customer/Supplier	Catalogue validates JWT tokens via the Auth Service before permitting admin mutations
Identity & Access	Lending	Customer/Supplier	Lending resolves user identity and role before issuing a loan
Identity & Access	Book Club	Customer/Supplier	Book Club verifies membership privileges through the Identity context
Catalogue	Lending	Published Language	Lending references Book Copy identifiers via a shared DTO contract
Lending	Notification	Event-Driven	Lending emits domain events (LoanCreated, LoanOverdue) that Notification subscribes to
Book Club	Notification	Event-Driven	Club events (MemberJoined, MessagePosted) trigger Notification events

2. DDD Modelling within Each Context

Each bounded context is made up of all the DDD parts: Entities, Value Objects, Aggregates (and their roots), Repositories, Domain Services, and Domain Events.

2.1 Identity & Access Context

Aggregate: UserAccount

In this case, the UserAccount aggregate is the only one. It contains rules for user identity and access.

DDD Construct	Name	Description
Aggregate Root (Entity)	UserAccount	The main entity that owns the lifecycle of identity data. UserId (UUID), email (Email VO), passwordHash (String), fullName (String), role (Role VO), and createdAt (DateTime) are the fields.
Value Object	Email	Email address that can't be changed. Makes sure the format stays the same. No identity.
Value Object	Role	Enum value: STUDENT STAFF ADMIN. Once set by admin, it is immutable.
Value Object	HashedPassword	Encapsulates bcrypt hash. Exposes verify(plaintext): Boolean. Never exposes raw hash.
Domain Service	AuthenticationService	Verifies credentials, issues JWT. Operates across UserAccount and JWT infrastructure. Stateless.
Domain Service	TokenValidationService	Validates incoming JWT tokens; extracts Role and userId claim for downstream contexts.
Repository	UserAccountRepository	Persistence abstraction: findByEmail, findById, save, delete.
Domain Event	UserRegistered	When a new UserAccount is saved, this is fired. Payload: email, userId, and role.
Domain Event	PasswordChanged	Fired on successful password reset. Payload: userId, timestamp.

Invariants (Identity & Access)

- ☐ Every UserAccount instance must have a different email address.
- ☐ AuthenticationService makes sure that only an existing ADMIN can give someone a STAFF or ADMIN role.

- ❑ Never let anyone see HashedPassword outside of the UserAccount aggregate boundary.
- ❑ JWT expiration is a system-level non-functional constraint. The TokenValidationService makes sure that expired tokens don't get to domain logic.

2.2 Catalogue Context

Aggregate: BookCopy

BookCopy is the transactional aggregate, which means it's one copy, either physical or digital. Book is a separate entity that gives descriptive information and is owned by Catalogue.

DDD Construct	Name	Description
Aggregate Root (Entity)	BookCopy	Represents a borrowable copy. Fields: copyId (UUID), bookId (ref), availability (AvailabilityStatus VO), condition (String).
Entity	Book	Descriptive aggregate for a title. Fields: bookId (UUID), isbn (ISBN VO), title, description, publicationYear, totalCopies, availableCopies.
Entity	Author	authorId (UUID), name. Associated to Book (0..*).
Entity	Category	categoryId (UUID), name. Tag-style; a Book may have 0..* Categories.
Value Object	ISBN	Immutable 10 or 13-digit validated identifier. Format enforced at construction.
Value Object	AvailabilityStatus	Enum: AVAILABLE BORROWED RESERVED. Domain rules protect transitions
Domain Service	BookSearchService	Search books by title, author, category, and availability. Returns read-model projections.
Domain Service	InventoryService	Adjusts availableCopies on Book when a BookCopy changes AvailabilityStatus.
Repository	BookRepository	findByid, findByisbn, search(criteria), save, delete.

DDD Construct	Name	Description
Repository	BookCopyRepository	findAvailableCopiesByBookId, findById, save.
Domain Event	BookAdded	Fired when a new Book is created by an Admin. Payload: bookId, isbn, title.
Domain Event	CopyAvailabilityChanged	Fired when the status of a BookCopy changes. Payload: copyId, bookId, and newStatus. Lending took over.

Invariants (Catalogue)

- ☐ availableCopies on Book must equal the count of BookCopy instances with AvailabilityStatus = AVAILABLE.
- ☐ Every Book entity must have a different ISBN.
- ☐ A BookCopy can't go straight from BORROWED to RESERVED; it has to go through AVAILABLE first.
- ☐ At all times, totalCopies must be $\geq$ availableCopies.

2.3 Lending Context

Aggregate: Loan

The loan is the central part of the borrow-and-return process. It only uses the IDs of BookCopy and UserAccount to refer to them (no cross-context object references).

DDD Construct	Name	Description
Aggregate Root (Entity)	Loan	Fields: loanId (UUID), userId (ref), copyId (ref), issueDate (Date), dueDate (DueDate VO), returnDate (Date [0..1]), status (LoanStatus VO).
Value Object	DueDate	Immutable. Calculated at issuance (issueDate + 14 days default). Exposes isOverdue(): Boolean.
Value Object	LoanStatus	Enum: ACTIVE RETURNED OVERDUE. Transitions enforced by Loan aggregate.

DDD Construct	Name	Description
Domain Service	LoanIssuanceService	Checks to see if a user is eligible and if a copy is available (through an ACL/API call to the Catalogue). Makes a Loan and fires LoanCreated.
Domain Service	ReturnProcessingService	Processes return: sets returnDate, changes LoanStatus to RETURNED, publishes CopyReleased event.
Domain Service	OverdueDetectionService	Scheduled service: checks active Loans where DueDate.isOverdue() = true, changes status to OVERDUE, and sends LoanOverdue.
Repository	LoanRepository	findById, findActiveByUserId, findOverdue, save.
Domain Event	LoanCreated	Payload: loanId, userId, copyId, dueDate. Consumed by Notification.
Domain Event	LoanOverdue	Payload: loanId, userId, copyId, dueDate. Consumed by Notification.
Domain Event	CopyReleased	Payload: copyId. Consumed by Catalogue to restore AVAILABLE status.

Invariants (Lending)

- ☐ LoanIssuanceService ensures that a user can't have more than five active loans at the same time.
- ☐ A Loan cannot be issued for a BookCopy with AvailabilityStatus != AVAILABLE.
- ☐ returnDate must be >= issueDate.
- ☐ Once LoanStatus = RETURNED, the Loan is immutable.
- ☐ The system actor OverdueDetectionService sets LoanStatus = OVERDUE, never a user action.

2.4 Notification Context

Aggregate: Notification

Notification is a lightweight context. Its aggregate is a record of a notification that was sent out, made up of domain events from Lending and Book Club that came in.

DDD Construct	Name	Description
Aggregate Root (Entity)	Notification	Fields: notificationId (UUID), userId (ref), type (NotificationType VO), message (String), createdAt (DateTime), sentAt (DateTime [0..1]), status (NotificationStatus VO).
Value Object	NotificationType	Enum: DUE_SOON_REMINDER OVERDUE_ALERT BORROW_CONFIRMATION RETURN_CONFIRMATION CLUB_ANNOUNCEMENT.
Value Object	NotificationStatus	Enum: PENDING SENT FAILED. Transitions guarded by dispatch logic.
Domain Service	NotificationDispatchService	Creates Notification entities from incoming events. Ways to get to the delivery channel (email or in-app). Marks SENT or FAILED.
Repository	NotificationRepository	findById, findByUserId, findPending, save.
Domain Event (consumed)	LoanCreated (from Lending)	Triggers BORROW_CONFIRMATION notification.
Domain Event (consumed)	LoanOverdue (from Lending)	Triggers OVERDUE_ALERT notification 3 days before and on due date.

Invariants (Notification)

- ☐ You have 30 seconds to send a BORROW_CONFIRMATION notification after you get a LoanCreated event.
- ☐ A DUE_SOON_REMINDER can only be sent once per loan per trigger window (3-day mark).
- ☐ Notifications that fail are retried again up to three times before they are marked as permanently failed.

2.5 Book Club Context

Aggregates: BookClub, ClubMembership, Message

DDD Construct	Name	Description
---------------	------	-------------

Aggregate Root (Entity)	BookClub	clubId (UUID), name (String), description (String [0..1]), createdAt (DateTime). Owns ClubMembership collection.
Entity	ClubMembership	membershipId (UUID), userId (ref), clubId (ref), role (ClubRole VO), joinedAt (DateTime). Child of BookClub aggregate.
Aggregate Root (Entity)	Message	messageId (UUID), content (MessageContent VO), clubId (ref), authorUserId (ref), sentAt (DateTime), editedAt (DateTime [0..1]).
Value Object	ClubRole	Enum: OWNER MODERATOR MEMBER.
Value Object	MessageContent	Immutable string; enforces max 2000 character limit and non-empty invariant.
Domain Service	ClubModerationService	Handles member role changes, member removal. Only OWNER or MODERATOR may invoke.
Domain Service	MessageService	Handles post/edit/delete of Messages. Edit sets editedAt; delete is soft-delete. Only author or MODERATOR may delete.
Repository	BookClubRepository	findById, findAll, save, delete.
Repository	MessageRepository	findByClubId(paginated), findById, save.
Domain Event	MemberJoined	Payload: clubId, userId. Consumed by Notification for welcome message.
Domain Event	MessagePosted	Payload: messageId, clubId, authorUserId. May trigger club-level notification.

Invariants (Book Club)

- ☐ Each BookClub must have exactly one OWNER (ClubRole = OWNER) at all times.
- ☐ An OWNER cannot leave a club unless they first transfer ownership to another MEMBER.
- ☐ MessageContent must be non-empty and no longer than 2000 characters.
- ☐ A user may join a given BookClub at most once (unique userId per clubId in ClubMembership).

3. Aggregate Design & Invariants

This section explains why the aggregate boundary decisions were made and shows how the system's transactional consistency rules work.

3.1 Aggregate Boundary Justifications

Aggregate Root	Boundary Justification
UserAccount	At the same time, all identity fields, such as the password, email, and role, are changed. These fields don't need to be changed in any other context.
BookCopy	BookCopy is the unit that tracks availability. A book can have many copies, and the status of each copy can change on its own without locking the whole book.
Book	The Book group is separate because the descriptive data (title, ISBN, author list) changes on its own, no matter if a certain copy is available or not.
Loan	A Loan shows you how to borrow money from start to finish. There is only one transaction for all changes in state (ACTIVE -> OVERDUE -> RETURNED) on this aggregate.
Notification	Every Notification is sent out by itself. You don't have to lock a group of notifications in a transaction.
BookClub	A BookClub and its ClubMembership collection are modified together, adding/removing members is an operation on the club. This justifies a single aggregate.
Message	Messages are independent from the BookClub aggregate (a club can have thousands of messages). Separating them prevents contention on the BookClub aggregate root.

3.2 Invariant Summary

Context	Invariant	Enforcement Point
Identity & Access	Email uniqueness across all UserAccount instances	UserAccountRepository + domain precondition check

Context	Invariant	Enforcement Point
Identity & Access	Role elevation requires ADMIN actor	AuthenticationService
Catalogue	availableCopies == count(BookCopy[AVAILABLE])	InventoryService on CopyAvailabilityChanged event
Catalogue	ISBN uniqueness	BookRepository precondition
Catalogue	BORROWED - > AVAILABLE only (no direct RESERVED)	AvailabilityStatus VO transition guard
Lending	Max 5 active loans per user	LoanIssuanceService precondition
Lending	Loan requires AVAILABLE copy	LoanIssuanceService via Catalogue ACL call
Lending	RETURNED loans are immutable	Loan entity state guard
Notification	DUE_SOON_REMINDER sent at most once per loan per window	NotificationRepository deduplication query
Notification	Max 3 retries on FAILED	NotificationDispatchService retry counter
Book Club	Exactly 1 OWNER per BookClub at all times	ClubModerationService ownership transfer guard
Book Club	User joins club at most once	ClubMembershipRepository unique constraint + domain check
Book Club	MessageContent: 1–2000 chars	MessageContent Value Object constructor

4. Inter-Context Contracts

A written contract governs every interaction between different contexts. This section discusses how the API/event contracts, the Anti-Corruption Layer strategies, and what each side of the contract must do.

4.1 REST API Contracts (Synchronous)

Contract	Upstream	Downstream	Endpoint	Response DTO	ACL / Strategy
Token Validation	Identity & Access	All Contexts (via Gateway)	GET /auth/validate	{ userId, role, valid: bool }	Gateway intercepts; downstream receives validated claims only, never raw JWT internals
Copy Availability Check	Catalogue	Lending	GET /catalogue/copies/{copyId}/availability	{ copyId, status, bookTitle }	Lending ACL maps CatalogueAvailabilityDTO - internal CopyStatusVO, avoids Catalogue model leaking into Lending
User Profile Read	Identity & Access	Book Club	GET /users/{userId}/profile	{ userId, fullName, role }	Book Club ACL maps to internal ClubMemberProfileVO, only the fields needed are extracted

4.2 Domain Event Contracts (Asynchronous)

Event	Publisher	Subscriber	Payload Schema	Handling Strategy
LoanCreated	Lending	Notification	{ loanId, userId, copyId, bookTitle, dueDate, timestamp }	Notification builds BORROW_CONFIRMATION. Idempotent: loanId used as deduplication key.
LoanOverdue	Lending	Notification	{ loanId, userId, copyId, bookTitle, dueDate, daysOverdue, timestamp }	Notification builds OVERDUE_ALERT. Deduplication by loanId + notification type.
CopyReleased	Lending	Catalogue	{ copyId, loanId, returnedAt }	Catalogue ACL translates to AvailabilityStatus = AVAILABLE transition on BookCopy.
MemberJoined	Book Club	Notification	{ clubId, clubName, userId, joinedAt }	Notification builds CLUB_ANNOUNCEMENT for the joining user.
UserRegistered	Identity & Access	Notification	{ userId, email, fullName, timestamp }	Notification sends welcome message (optional, BORROW_CONFIRMATION type).

4.3 Anti-Corruption Layer Design

An ACL is used by each downstream context to keep its domain model safe from changes made upstream. Before any domain logic is called, the ACL sits at the edge of the downstream context and does the translation.

Downstream Context	ACL Class	Purpose
Lending	CatalogueACL	Translates Catalogue REST DTO -> Lending's internal CopyReference value object. If

Downstream Context	ACL Class	Purpose
		Catalogue changes its DTO, only CatalogueACL is updated.
Catalogue	LendingEventTranslator	Translates CopyReleased event payload -> AvailabilityUpdateCommand for the Catalogue domain service.
Notification	LendingEventTranslator	Translates LoanCreated/LoanOverdue -> NotificationRequest internal model.
Notification	ClubEventTranslator	Translates MemberJoined/MessagePosted - NotificationRequest internal model.
Book Club	IdentityACL	Translates User profile REST response - ClubMemberProfileVO used only within the Book Club context.

5. Conformance to UML/DDD Notation

All UML models made for this assignment use the same notation standard, which comes from DDD modelling conventions. This section talks about the notation guide that was used and checks the readability standards.

5.1 Ubiquitous Language Glossary

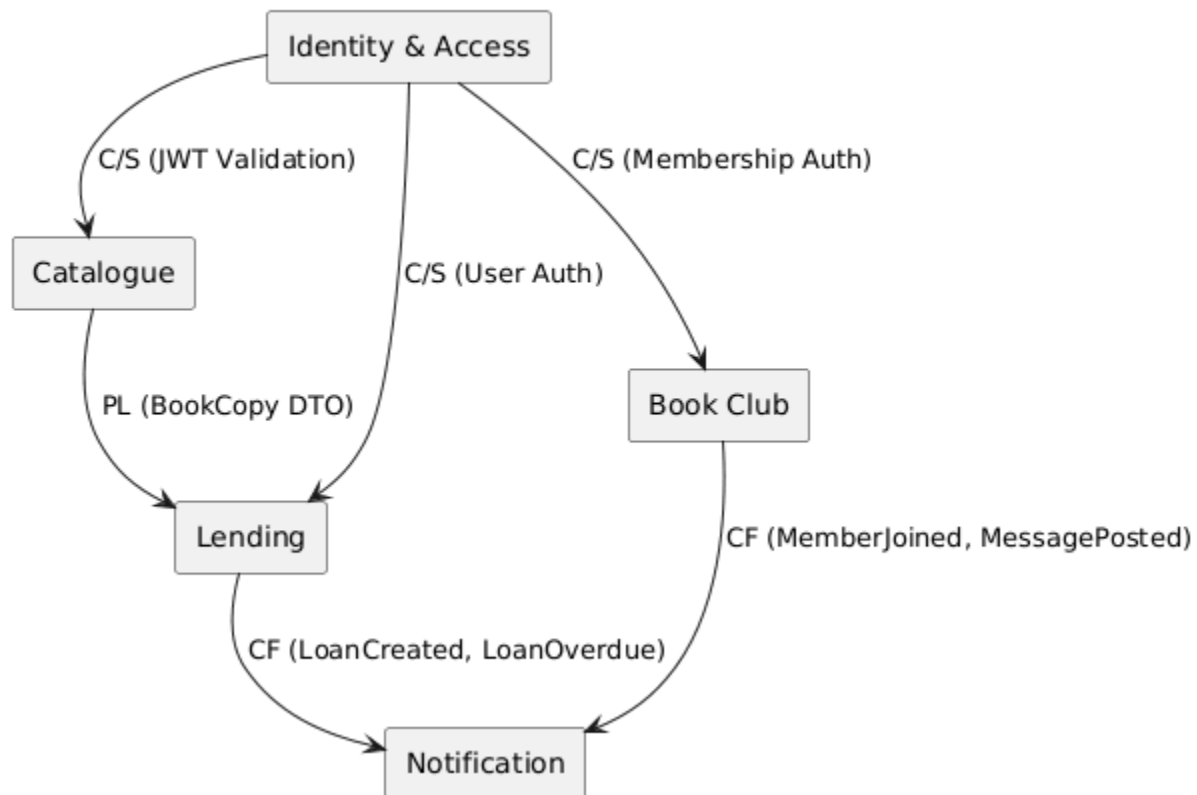
All UML models made for this assignment use the same notation standard, which comes from DDD modelling conventions. This section talks about the notation guide that was used and checks the readability standards.

Term	Context(s)	Definition
Loan	Lending	A record representing a user borrowing one BookCopy, from issueDate to returnDate.
BookCopy	Catalogue / Lending	A single borrowable instance of a Book title (physical or digital).
Book	Catalogue	The descriptive metadata for a title: ISBN, title, authors, categories.
UserAccount	Identity & Access	The domain entity representing a registered user with credentials and a Role.
Role	Identity & Access	An enum attribute of UserAccount: STUDENT, STAFF, or ADMIN.
DueDate	Lending	A Value Object representing the date by which a Loan must be returned.
ClubMembership	Book Club	The association between a UserAccount and a BookClub, with a ClubRole.
Notification	Notification	A dispatched or queued message sent to a user about a system event.
AvailabilityStatus	Catalogue	The current borrowability state of a BookCopy: AVAILABLE, BORROWED, or RESERVED.
ACL (Anti-Corruption Layer)	All	A translation layer at a context boundary preventing model pollution from upstream contexts.
Published Language	All	A versioned, shared DTO/event schema agreed between producer and consumer.

Term	Context(s)	Definition
Domain Event	All	An immutable record of something that occurred in the domain, named in past tense.

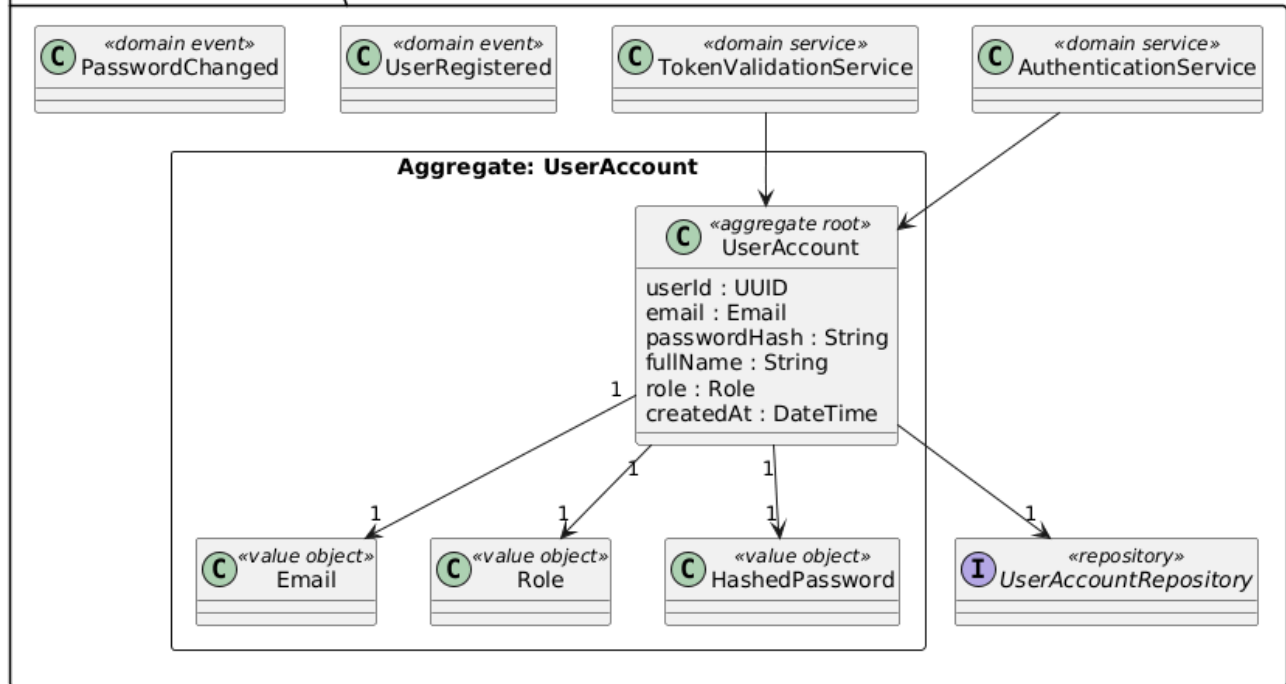
5.2 UML models

Context Map Diagram (v1.0)



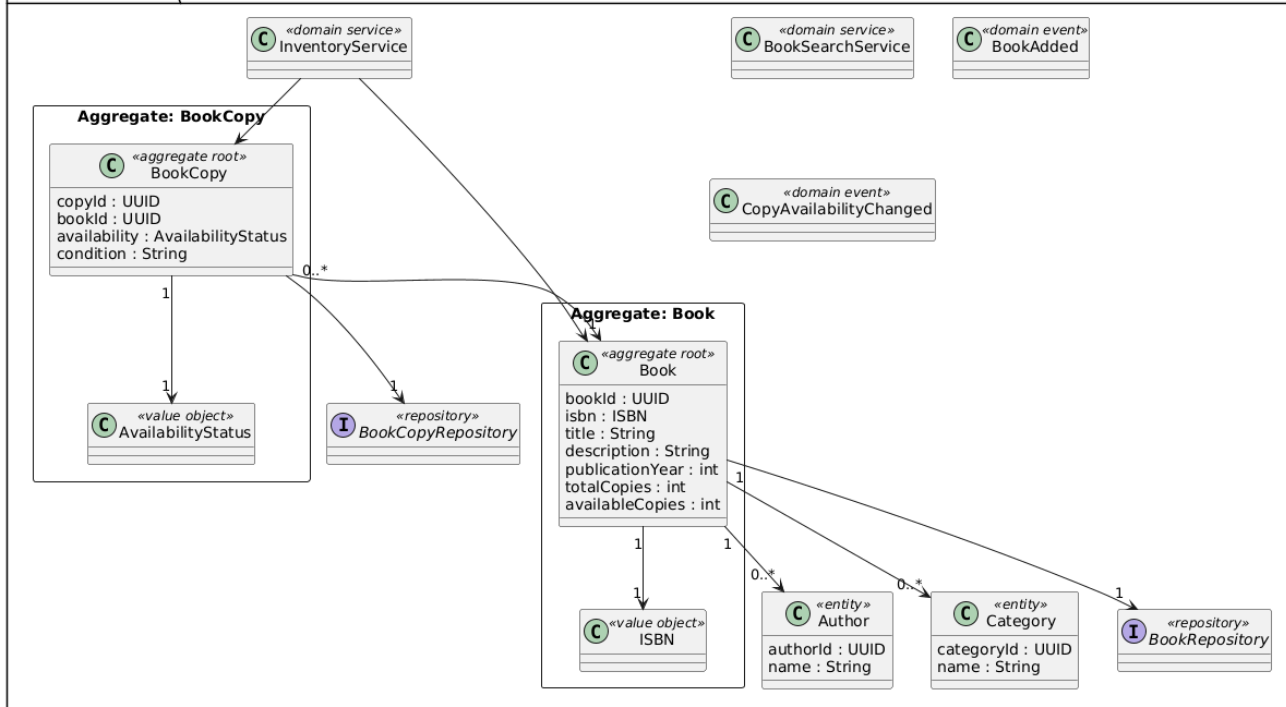
Identity & Access Context - Class Diagram (v1.0)

Identity & Access Context

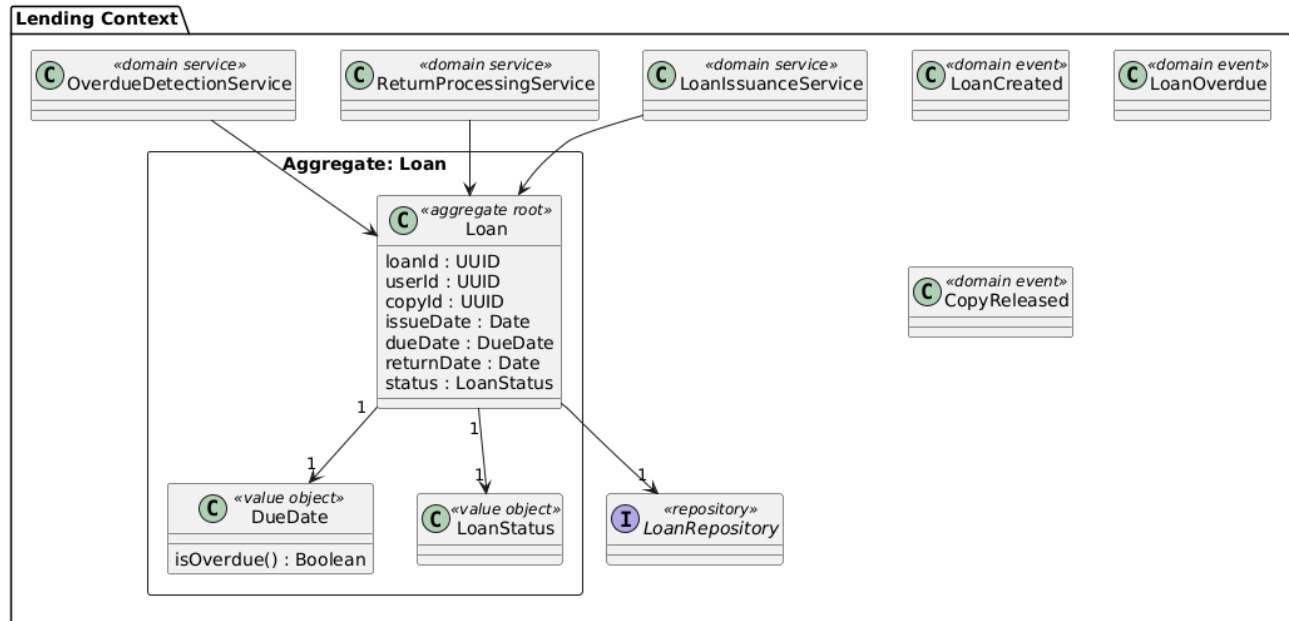


Catalogue Context - Class Diagram (v1.0)

Catalogue Context

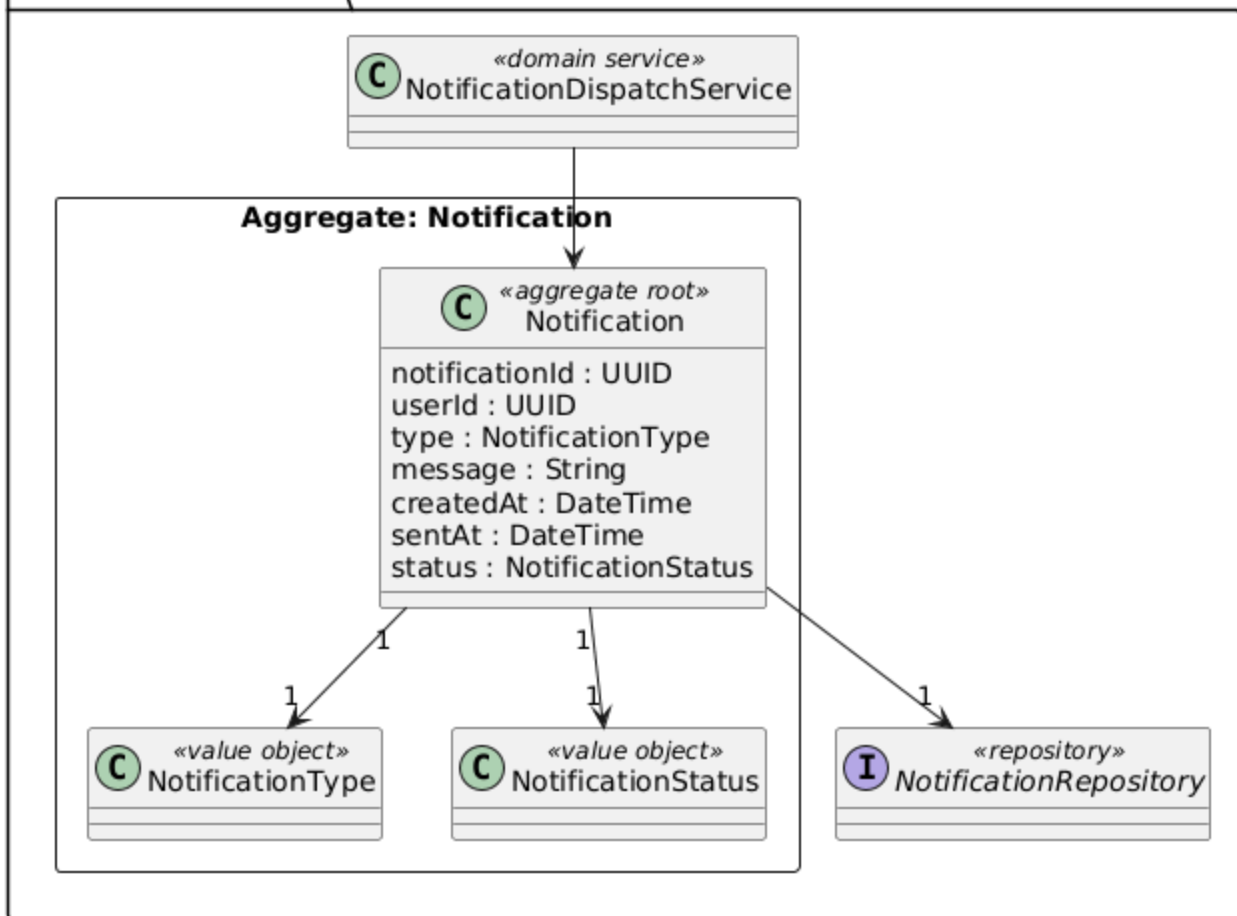


Lending Context - Class Diagram (v1.0)

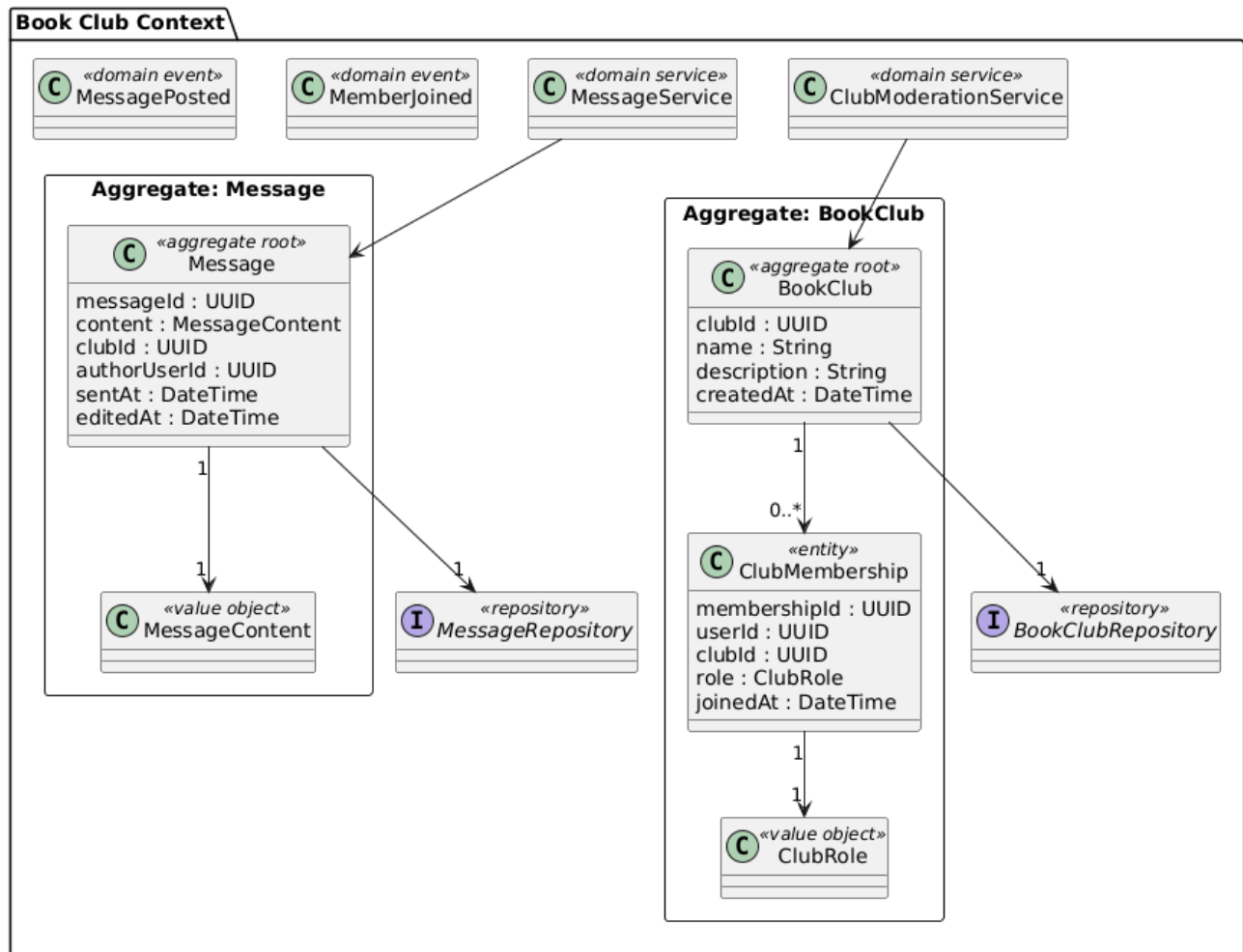


Notification Context - Class Diagram (v1.0)

Notification Context



Book Club Context - Class Diagram (v1.0)



6. API Gateway

The E-Library system has a central API Gateway that all client requests pass through. Instead of allowing external users see each bounded context directly, every incoming HTTP request goes through the gateway first. This method makes sure that security boundaries are always the same, separates clients from the internal service topology, and gives you one place to deal with things like authentication and routing that affect many parts of the system.

6.1 Gateway Responsibilities

The gateway does the following things before sending any request to a downstream bounded context:

Before sending any request to a downstream bounded context, the gateway does the following:

JWT Validation (JWT Filter): The gateway uses a dedicated JWT authentication filter (e.g. `OncePerRequestFilter` in Spring Security) to intercept every incoming HTTP request to protected routes. The filter extracts the Bearer token from the Authorisation header and validates it locally using a shared HMAC-SHA secret key. If the token is missing, invalid, or expired, the filter immediately terminates the request and returns a 401 Unauthorised response. Requests to public authentication paths (`/api/user/auth/**`) bypass this filter.

Role Verification: The gateway checks the token and gets the role claim. It then checks that it is one of the three system roles that are known (STUDENT, STAFF, ADMIN). If a role is missing or not recognised, a 401 Unauthorised error occurs.

The JWT filter operates within the gateway's security filter chain, ensuring that authentication is enforced consistently before any request is routed to downstream services.

Path prefix	Downstream service
<code>/api/user/</code>	elib-identity
<code>/api/book-catalogue/</code>	elib-catalogue
<code>/api/lending/</code>	elib-lending
<code>/api/notifications/</code>	elib-notification
<code>/api/bookclub/</code>	elib-bookclub

6.2 Request Lifecycle

The following sequence shows how a normal authenticated request moves through the system:

Receive: The client makes an HTTP request to the gateway endpoint.

Public path check: If the request path starts with `/api/user/auth`, it is sent on without checking for a token. This means that login and registration calls can go through without being verified.

Authenticate (JWT Filter Execution): The request processing pipeline starts the JWT authentication filter. It extracts the Bearer token from the Authorisation header and checks it against the shared HMAC secret. If the token is missing, broken, or expired, the filter stops the request and sends back a 401 Unauthorised response before it gets to any routing logic.

Check the role: The gateway gets the role claim from the parsed token. If the claim is missing or doesn't match STUDENT, STAFF, or ADMIN, a 401 Unauthorised message is sent back.

Route: The validated JWT claims are added to the request as a servlet attribute. The gateway gets the target service instance from Eureka and sends the request to the right bounded context. The raw Authorisation header is sent along as-is, and downstream services get the original request without any claim headers being added.

Respond: The response from the downstream context is sent back to the client without any changes.

6.3 Relationship to the Cross-Context Dependency Map

The gateway centralises JWT validation, which means that all incoming requests must be authenticated before they can reach any bounded context. The gateway checks the request locally using a shared secret instead of calling the Identity context at runtime. This means that the Identity service is not involved in every request and does not become a synchronous bottleneck. But downstream services that need to know who is calling must re-parse the Authorisation header themselves instead of using pre-extracted values from the gateway because validated claims are not sent as headers.

Part B: Event Storming & Domain Storytelling

1. Event Storming Outcomes

Using a shared virtual Miro board, three Event Storming sessions were conducted. The full business timeline was mapped (Session 1), and this was followed by Sessions 2 & 3, which were Process-level sessions (Sessions 2 & 3) focusing on Lending and Book-Club sub-domains. All artefacts are colour-coded using the standard Event Storming palette, which is shown below.

1.1 Notation Key Used on the Board

Domain Event (orange)	Command (blue)	Actor / User (yellow)	Policy / Rule (purple/lilac)	Read Model / View (green)	External System (pink)	Aggregate (pale yellow)	Hotspot Issue (red)
---------------------------------	--------------------------	---------------------------------	--	-------------------------------------	----------------------------------	-----------------------------------	-------------------------------

1.2 Big-Picture Event Flow

The board was filled left-to-right in a chronological order. Each E-library pipeline is represented by a row. Commands, actors, policies, and hotspots are shown and are cross referenced to their respective events.

#	Domain Event (orange)	Triggering Command (blue)	Actor (yellow)	Policy / Rule (purple)	Hotspot / Conflict (red)
1	UserRegistered	Register User	Student / Staff	Email must be unique	
2	UserAuthenticated	Login	Any Role	Invalid credentials -> retry with backoff	How many retry attempts before lockout?
3	PasswordChanged	Change Password	Authenticated User	Must confirm current password first	
4	BookAdded	Add Book	Admin	ISBN must be unique; Admin role required	Who validates ISBN format - domain or UI?
5	BookUpdated	Update Book Details	Admin	Cannot reduce totalCopies	Concurrent update conflict possible

				below active loans	
6	BookSearched	Search Books	Student / Staff	Filter by title, author, category, availability	

7	CopyAvailabilityChecked	Check Availability	Student	Real-time copy count must be shown	Cache staleness - how fresh is availability data?
8	LoanCreated	Borrow Book	Student / Staff	Max 5 active loans; copy must be AVAILABLE	Race condition if two users borrow last copy simultaneously
9	DueDateAssigned	(system - on Borrow)	System	Due date = issueDate + 14 days by default	Should Staff have a different loan period?
10	BorrowConfirmationSent	(triggered by policy)	Notification Svc	On LoanCreated -> send confirmation within 30 secs	
11	DueSoonReminderSent	(scheduled trigger)	Notification Svc	3 days before due date; send once per loan per window	What channel - email, in-app, or both?
12	LoanOverdue	(scheduled trigger)	System Scheduler	On due date if not returned -> flag OVERDUE	No fine system currently - is one needed?
13	OverdueAlertSent	(triggered by policy)	Notification Svc	On LoanOverdue -> notify user and staff	
14	BookReturned	Return Book	Student / Staff	returnDate >= issueDate; set LoanStatus = RETURNED	

15	CopyReleased	(triggered by policy)	System	On BookReturned - > restore copy AvailabilityStatus	
16	ReturnConfirmationSent	(triggered by policy)	Notification Svc	On BookReturned - > confirm return to user	
17	BookClubCreated	Create Book Club	Any Authenticated User	Creator becomes OWNER automatically	
18	MemberJoined	Join Club	Student / Staff	Cannot join same club twice	Should clubs be moderated/inviteonly?
19	MessagePosted	Post Message	Club Member	Content <= 2000 chars; must be club member	Real-time delivery vs polling - which approach?
20	MemberRemoved	Remove Member	OWNER / MODERATOR	OWNER cannot remove themselves	
21	BookDeleted	Delete Book	Admin	Cannot delete if active loans exist for this book	Should deletion be soft-delete?

1.3 Hotspots & Conflicts Register

Using red stickies, certain hotspots and conflicts were raised during the session. These sessions were marked for resolution. In the table below, each is tagged with its resolution status.

ID	Hotspot Description	Context	Resolution / Decision	Status
H-01	Race condition: two users borrow the last available copy at the same time	Lending	LoanIssuanceService uses optimistic locking on the BookCopy aggregate; The second request gets HTTP 409 Conflict response with user-friendly	RESOLVED

			message	
H-02	Cache staleness: availability data shown to user may be out of date by seconds	Catalogue	Accept eventual consistency for search results; final availability check is done in real-time at time of borrowing (synchronous ACL call)	RESOLVED
H-03	Retry logout: no policy defined for repeated failed login attempts	Identity & Access	Add domain policy: 15 minute account lock if 5 failed attempts within 10 minutes. Added to backlog as BACK-07	RESOLVED
H-04	ISBN validation: should the domain or UI layer validate ISBN format?	Catalogue	Domain validates via ISBN Value Object constructor. Prevalidation UI provides client-side for UX only. Domain is authoritative.	RESOLVED
H-05	Concurrent book update: admin updates totalCopies while borrow is in flight	Catalogue / Lending	Domain invariant: totalCopies >= activeLoans enforced in InventoryService. Surface written as optimistic by conflicting as optimistic locking exception.	RESOLVED
H-06	Staff loan period: should Staff get a longer default loan period than Students?	Lending	Product decision: Staff gets 28-day default; Student 14-day default. Role parameter accepted by DueDate VO factory method. Added to backlog BACK-08	RESOLVED

H-07	Notification channel: email only, in-app only, or both?	Notification	Initial release: notifications in-app only. Email integration deferred to v2. Backlog item BACK-09 created.	DEFERRED
H-08	Fine system: should overdue books incur a penalty / fine?	Lending	Out of scope of project. Recorded as future feature BACK-10.	DEFERRED
H-09	Book club moderation: should clubs be openjoin, or invite/approval required?	Book Club	Default open-join. OWNER can toggle club to invite-only mode. Added to backlog BACK-11.	DEFERRED
H-10	Real-time messaging: WebSocket vs polling for club chat	Book Club	WebSocket (STOMP over SockJS) chosen for real-time feel. Event driven style used by Chat service internally. Added to backlog BACK-12.	RESOLVED
H-11	Soft delete: should Book deletion be hard or soft (archive)?	Catalogue	Soft-delete chosen: deletedAt timestamp added to Book entity. Loan history references remain valid. Added to domain model.	RESOLVED

1.4 Policies Discovered

Purple stickies, which represent policies, are automatic responses to events within the domain. For example: Do Y, when X happens. After discussions in the session, these policies are now written down as domain rules or application-level handlers.

Policy ID	Trigger Event	Policy Statement	Implemented In
POL-01	LoanCreated	Immediately send a BorrowConfirmationSent notification to the borrowing user when a loan is created.	NotificationDispatchService (async event handler)

POL-02	LoanCreated	When a Loan is created, mark the corresponding BookCopy as BORROWED (AvailabilityStatus)	LoanIssuanceService (synchronous, within Lending context)
POL-03	Scheduled (daily)	Each day, check all active Loans: if dueDate is exactly 3 days away and reminder not yet sent, send DueSoonReminderSent	OverdueDetectionService (scheduled job, @Scheduled annotation)
POL-04	Scheduled (daily)	Each day, check all active Loans: transition LoanStatus to OVERDUE if dueDate has passed, and fire LoanOverdue	OverdueDetectionService (scheduled job)
POL-05	LoanOverdue	When LoanOverdue fires, dispatch an OverdueAlertSent notification to the user, and to Staff, a summary alert.	NotificationDispatchService (async event handler)
POL-06	BookReturned	When a Book is returned, set LoanStatus = RETURNED, set returnDate, and fire CopyReleased	ReturnProcessingService (synchronous, within Lending)
POL-07	CopyReleased	When a CopyReleased event is received in Catalogue, update BookCopy.AvailabilityStatus = AVAILABLE	CatalogueEventHandler (async, Catalogue context)
POL-08	BookReturned	When a Book is returned, dispatch a ReturnConfirmationSent notification	NotificationDispatchService (async event handler)
POL-09	UserRegistered	When a new user registers, send a welcome notification	NotificationDispatchService (async event handler)
POL-10	MemberJoined	When a user joins a Book Club, send a welcome club notification	NotificationDispatchService (async event handler)
POL-11	BookDeleted (Admin)	When an Admin deletes a Book, check for active loans, and reject command and return domain error if any exist.	BookManagementService (precondition guard)

2. Translation of Insights

The section below shows how (a) updated domain model elements, (b) policies, or (c) product backlog items are linked to the relevant stakeholder insight and hotspot resolution.

It shows how each significant stakeholder insight and hotspot resolution is connected to an updated domain model element, a policy, or a product backlog item. Each Delta is explained and commits reference the relevant GitHub issue or branch.

2.1 Model Changes Introduced by Event Storming

The table below shows every change to the domain model that was made during Event Storming sessions, compared to Assignment 3 Part A.

Delta ID	Change Type	Element Changed	Before (Part A Baseline)	After (Post-Event Storming)	Traceable To
D-01	New Value Object	DueDate (Lending)	Modelled as plain Date field on Loan	Extracted into DueDate VO; factory method accepts Role to set 14day (Student) or 28-day (Staff) window; exposes isOverdue(): Boolean and daysUntilDue(): Int	H-06, POL-03, POL-04
D-02	New Domain Event	CopyReleased (Lending ->Catalogue)	No explicit release event; availability updated synchronously inside Lending	Explicit CopyReleased domain event published by ReturnProcessingService ; Catalogue subscribes asynchronously via ACL	POL-07, B1 flow step 15
D-03	Invariant Added	Book.deletedAt (Catalogue)	Hard-delete: row removed from DB	Soft-delete: deletedAt DateTime [0..1] added to Book entity; for search, deleted books should be hidden but	H-11, POL-11

				preserved for loan history FK integrity	
D-04	New Policy Object	AccountLockPolicy (Identity & Access)	No lockout policy defined	New domain rule: LockoutPolicy VO encapsulates maxAttempts (5), windowMinutes (10), lockDurationMinutes (15); enforced in AuthenticationService	H-03, BACK-07
D-05	Aggregate Change	ClubMembership.inviteRequired (Book Club)	ClubMembership had no invite flag	BookClub gains inviteOnly: Boolean attribute; MemberJoined command path branches based on this flag; invite-only clubs, new MembershipRequestCreated event added for	H-09, BACK-11
D-06	New Domain Service	OverdueDetectionService (Lending)	No explicit scheduled service; described informally	Formalised as a Domain Service with clear interface: detectAndMarkOverdue(), sendRemindersBefore(days: Int); owns the LoanOverdue event publication	POL-03, POL-04, H-06
D-07	New Domain Event	MembershipRequestCreated (Book Club)	Did not exist	When user requests to join an invite-only club, a new event is published; OWNER/MODERATOR must approve via ApproveMembership command	D-05, H09

D-08	Multiplicity Update	Notification -> User association	0..* Notifications received by User (conceptual only)	Notification.userId is a reference key; NotificationRepository exposes findByUserId(paginated) for user inbox read model	B1 flow steps 10-13, 16
D-09	New Read Model	UserBorrowingHistory (Lending)	No explicit read model; raw Loan query only	Dedicated read-model projection: for efficient dashboard rendering, UserBorrowingHistory aggregates Loan records per userId; updated on LoanCreated/BookReturned events	B1 flow, Domain Storytelling DS-01
D-10	Invariant Tightened	Loan max active per user	Described as max 5 in SRS; not modelled as a VO	MaxActiveLoansPolicy VO (value: 5) added into LoanIssuanceService; configuration-driven so code change isn't needed for value change	H-08 (related), POL-02

2.2 Backlog Items Generated

A traceable backlog item was produced for each unresolved hotspot and every model delta that was produced. The table below shows critical items introduced or updated during the Event Storming sessions. The full backlog is in Appendix B.

Backlog ID	Title	Type	Priority	Origin
BACK-01	Implement optimistic locking on BookCopy to prevent double-borrow race condition	Bug / Invariant	HIGH	H-01
BACK-02	Build LoanIssuanceService with real-time copy availability check via CatalogueACL	Feature	HIGH	H-01, H-02
BACK-03	Implement OverdueDetectionService as a @Scheduled job in the Lending microservice	Feature	HIGH	POL-03, POL-04, D-06

BACK-04	Implement soft-delete for Books (add deletedAt, filter in search queries)	Feature	HIGH	H-11, D-03
BACK-05	Implement CopyReleased async event and Catalogue subscriber	Feature	HIGH	D-02, POL-07
BACK-06	Extract DueDate Value Object with role-based factory (14-day Student / 28-day Staff)	Refactor / Feature	HIGH	H-06, D-01
BACK-07	Implement AccountLockPolicy (5 attempts / 10-min window / 15-min lock)	Security Feature	HIGH	H-03, D-04
BACK-08	UserBorrowingHistory read-model projection for the dashboard	Feature	MEDIUM	D-09
BACK-09	Email notification channel integration (v2 scope)	Feature	LOW	H-07
BACK-10	Investigate fine/penalty system for overdue loans (future feature)	Research	LOW	H-08
BACK-11	Implement invite-only Book Club mode with membership request/approval flow	Feature	MEDIUM	H-09, D-05, D-07
BACK-12	Implement WebSocket (STOMP/SockJS) for real-time club chat	Feature	MEDIUM	H-10

3. Domain Storytelling

Using structured narratives, domain stories were used to capture how real actors collaborate to achieve goals in the domain. The format followed by each story: Actor -> [does something with] -> Work Object -> [to/from] -> Other Actor. These stories were created with all team members acting as domain expert proxies. This occurred during Session 3 and was validated against the event flow discovered in Session 1.

3.1 DS-01: Student Borrows a Book

Actors: Student, E-Library System, Lending Service, Notification Service

Trigger: Student wants to borrow an available book

Outcome: Loan record created; student receives confirmation; copy marked unavailable

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Authentication	Student	Navigates to the login page and university credentials submission	Login Form (email + password)	UserAccount, HashedPassword
2	Authentication	E-Library System	Validates credentials against the UserAccount aggregate; issues a JWT	JWT Token (contains userId, role=STUDENT)	AuthenticationService, JWT, Role
3	Authentication	E-Library System	Redirects student to their personalised User Dashboard	User Dashboard (shows active loans, search bar)	Read Model: UserBorrowingHistory
4	Book Search	Student	Types a book title into the search bar and submits	Search Query (title, category filter)	BookSearchService, Category
5	Book Search	Catalogue Service	Executes search; matching Book records with live availability counts returned	Search Results (title, author, availableCopies)	BookCopy, AvailabilityStatus
6	Book Selection	Student	Identifies the desired book and clicks 'Borrow'	Borrow Command (bookId, userId)	Command: BorrowBook
7	Availability	Lending Service	Calls CatalogueACL to verify that at least one BookCopy is AVAILABLE	CopyAvailabilityResponse (copyId, status=AVAILABLE)	CatalogueACL, AvailabilityStatus
8	Loan Creation	Lending Service	Creates Loan aggregate; assigns DueDate based on Role (Student = +14 days)	Loan (loanId, issueDate, dueDate, ACTIVE)	Loan, DueDate VO, LoanIssuanceService

9	Copy Update	Lending Service	Fires LoanCreated event; BookCopy updated to BORROWED by Catalogue ACL	CopyReleased / AvailabilityStatus = BORROWED	Domain Event: LoanCreated, Policy POL-02
10	Notification	Notification Service	Consumes LoanCreated event; dispatches BorrowConfirmation Sent to the student	In-app Notification (loan details, due date)	NotificationDispatchService, Policy POL-01
11	Confirmation	Student	Confirmation notification received in their dashboard; sees book in Active Loans	Updated Dashboard (loan listed, due date visible)	LoanStatus = ACTIVE, Read Model

Ubiquitous Language Improvements from DS-01

- 'Borrow' replaces 'issue' in all user-facing language, now students 'borrow' books, the system 'issues' loans internally. Both terms now have distinct definitions in the glossary.
- 'Due Date' is now always expressed as a Date (not DateTime) in the UI, the time component is hidden from users to avoid confusion.
- 'Available Copies' is the user-facing term; internally the domain uses AvailabilityStatus on individual BookCopy instances.

3.2 DS-02: Admin Manages Book Inventory

Actors: Admin, E-Library System, Catalogue Service

Trigger: New books have arrived; Admin must add them to the system

Outcome: Books available for search and borrowing; notifications sent if staff are watching

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Authentication	Admin	Logs in; JWT contains role=ADMIN	JWT (role=ADMIN)	Role, RBAC enforcement

2	Navigation	Admin	Opens the Admin Book Management panel	Admin Dashboard (restricted route)	RBAC protected route
3	Book Entry	Admin	Fills in book form: ISBN, title, description, authors, categories, copy count	AddBook Command (all fields)	Command: AddBook
4	ISBN Validation	Catalogue Service	ISBN VO constructor validates format (10 or 13 digits); domain rejects invalid	Domain Validation Result	ISBN Value Object, invariant
5	Persistence	Catalogue Service	Creates Book entity and n BookCopy instances; sets all copies of books to AVAILABLE	BookAdded Event (bookId, isbn, title, copies)	Domain Event: BookAdded, BookCopy
6	Inventory	Catalogue Service	InventoryService sets availableCopies = N; updates totalCopies	Book record (totalCopies=N, availableCopies=N)	InventoryService, invariant
7	Confirmation	E-Library System	Returns success response; admin book list adds book immediately	Updated Admin Book List	Read Model
8	Update flow	Admin	Later updates book description with Edit Book form	UpdateBook Command (bookId, new description)	Command: UpdateBook
9	Delete guard	Admin	Attempts to delete a book that has 2 active loans	DeleteBook Command (bookId)	Command: DeleteBook, POL-11
10	Reject	Catalogue Service	Domain check fails: active loans exist ->	Domain Error: 'Cannot delete	Invariant enforcement, soft-delete

			descriptive error accompanies rejected command	book with active loans'	
11	Soft delete	Admin	Chooses to archive the book instead; sets deletedAt	BookDeleted Event (soft) - book hidden from search	Domain Event: BookDeleted, D03

Ubiquitous Language Improvements from DS-02

- 'Archive' vs 'Delete': there is now a clear distinction between archiving (soft-delete, history preserved) and deletion. The term 'Archive Book' is added to the glossary.
- 'Book Copies' and 'Books' are now explicitly separate in all documentation, now users search for Books, but borrow Copies of said Book.

3.3 DS-03: Overdue Book - Reminder and Return Flow

Actors: System Scheduler, Student, Staff, Notification Service

Trigger: 3 days before due date, then due date passes without return

Outcome: Reminders sent; loan flagged overdue; staff alerted; book eventually returned

	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Scheduled scan	System Scheduler	OverdueDetectionService runs daily; queries Loans where dueDate = today + 3 days and reminder not yet sent	Loan query result (active loans approaching due date)	OverdueDetectionService, DueDate VO
2	Reminder trigger	System Scheduler	Qualifying Loan, fires DueSoonReminderService event for each	DueSoonReminder Sent Event (loanId, userId, dueDate)	Policy POL-03, idempotency check

3	Notification	Notification Service	Consumes event; dispatches reminder to the student in-app	In-app Notification: 'Your book is due in 3 days'	NotificationDispatchService, NotificationStatus
4	Student response	Student	Ignores the reminder (book not returned by due date)	(No action taken)	Actor inaction modelled as a domain trigger
5	Overdue scan	System Scheduler	OverdueDetectionService runs on due date; identifies the Loan as overdue	LoanOverdue Event (loanId, userId, dueDate)	Policy POL-04, LoanStatus = OVERDUE
6	Status update	Lending Service	Sets LoanStatus = OVERDUE on the Loan aggregate	Updated Loan (status=OVERDUE)	LoanStatus VO, Loan invariant
7	Alert	Notification Service	Consumes LoanOverdue event; sends OVERDUE_ALERT to student and Staff receive a summary	Two notifications: student alert + staff summary	Policy POL-05, NotificationType
8	Staff view	Staff	Logs in; views overdue loan report in Staff dashboard	Staff Dashboard - overdue loans list	Role = STAFF, readmodel
9	Return	Student	Later, app Return action returns the book	ReturnBook Command (loanId)	Command: ReturnBook
10	Processing	Lending Service	ReturnProcessingService: sets returnDate, LoanStatus = RETURNED, fires CopyReleased	BookReturned Event, CopyReleased Event	ReturnProcessingService, Policy POL-06
11	Availability	Catalogue Service	CatalogueEventHandler receives CopyReleased; updates BookCopy to AVAILABLE	BookCopy (status=AVAILABLE)	Policy POL-07, CatalogueACL
12	Confirmation	Notification Service	Dispatches ReturnConfirmation Sent notification to student	In-app Notification: 'Return confirmed'	Policy POL-08

Ubiquitous Language Improvements from DS-03

- 'Overdue' is now a formal LoanStatus value. It was used informally in descriptions but not modelled previously. The single point of truth for this transition is OverdueDetectionService.
- 'Reminder' vs 'Alert': Both terms are now distinct in the ubiquitous language. A Reminder (DUE_SOON_REMINDER) is proactive and friendly in tone; an Alert (OVERDUE_ALERT) is reactive and urgent in tone.
- The term 'return' is now consistently used for the actor action; the system internally says 'CopyReleased' - both terms coexist with clear ownership.

3.4 DS-04: Book Club - Create, Join, and Discuss

Actors: Authenticated User, Club Owner, Moderator, Notification Service

Trigger: A student wants to discuss a book with peers

Outcome: Club created; members join; real-time messages exchanged

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Create club	Authenticated User	Navigates to Book Clubs section; clicks 'Create Club'; fills in name and description	CreateClub Command (name, description, inviteOnly flag)	Command: CreateClub
2	Club creation	Book Club Service	Creates BookClub entity; creates ClubMembership for creator with ClubRole = OWNER	BookClubCreated Event (clubId, name, ownerId)	Domain Event: BookClubCreated, ClubRole = OWNER
3	Discovery	Another Student	Searches for clubs; finds the new club; clicks 'Join'	JoinClub Command (clubId, userId)	Command: JoinClub
4	Open join	Book Club Service	Club is open-join (inviteOnly=false); creates	MemberJoined Event (clubId, userId)	ClubMembership, Policy: cannot join twice

			ClubMembershi p with ClubRole = MEMBER		
5	Welcome notificatio n	Notification Service	Consumes MemberJoine d event; new member gets sent welcome club notification	In-app Notification: 'Welcome to [Club Name]'	Policy POL-10
6	Discussio n	Club Member	Opens club discussion page; types a message; submits	PostMessage Command (clubId, content)	Command: PostMessage, MessageContent VO
7	Message persist	Book Club Service	MessageServic e validates content (<= 2000 chars, non-empty); persists Message	MessagePosted Event (messagId, clubId, authorId)	Domain Event: MessagePosted, MessageService
8	Real-time	E-Library System	WebSocket (STOMP) pushes message to all connected club members in real-time	WebSocket push to club subscribers	Chat Service, eventdriven style
9	Moderatio n	Club Owner	Spots inappropriate message; uses Remove Message action	DeleteMessage Command (messagId)	ClubModerationServic e, soft-delete
10	Role change	Club Owner	Promotes an active member to MODERATOR role	ChangeRole Command (membershipId, newRole=MODERATO R)	ClubRole, ClubModerationServi ce

Ubiquitous Language Improvements from DS-04

- 'Book Club' is the clear primary term not 'Reading Group' or 'Discussion Forum'. Book Club is described as a user-created community centred on a shared reading interest. The glossary is updated to reflect this.
- 'Owner' and 'Moderator' are now formal ClubRole values, and are used consistently in UI labels, API responses, and domain documentation.
- 'Discussion' refers to all Messages within a club. Internally represented as Message entities, which the UI shows on the 'Discussion' tab

4. Facilitation & Evidence

In this section is documented the facilitation approach, session records, participant lists, decision log. As well as inks to the boards providing the evidence for the Event Storming and Domain Storytelling outputs above.

4.1 Session Summary

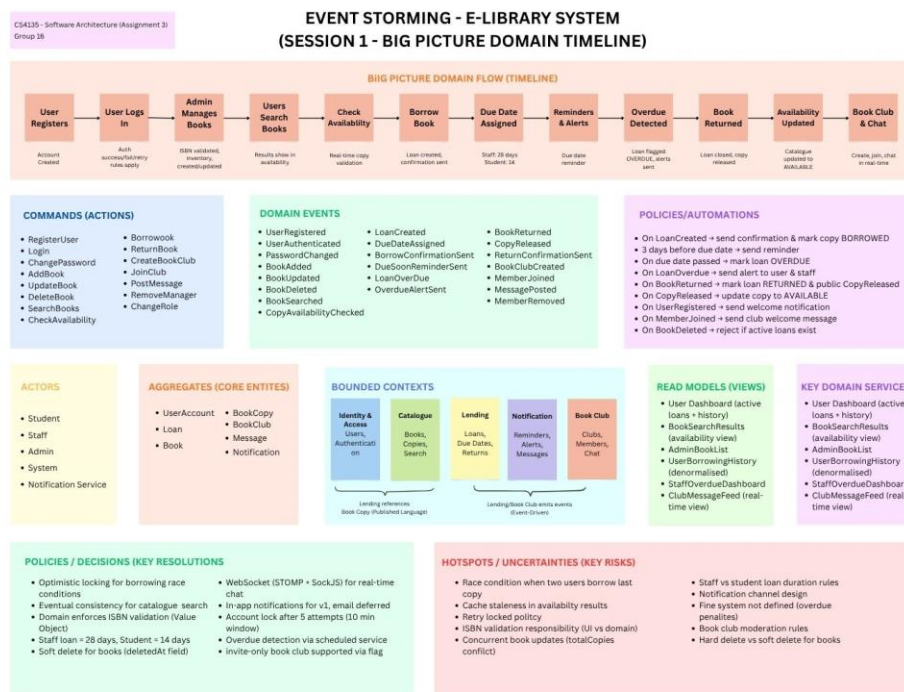
Session	Date	Duration	Format	Facilitator	Focus
Session 1 - Big Picture Event Storming	Week 8, Tuesday	2.5 hours	Virtual Miro Board	Oscar Zhou	Full domain timeline; event identification; hotspot tagging
Session 2 - ProcessLevel: Lending	Week 8, Thursday	1.5 hours	Virtual Miro Board	Caylum Hurley	Borrow/return/overdue flow; LoanIssuanceService design; H-01, H-02, H-06 resolution
Session 3 - Domain Storytelling	Week 9, Tuesday	2 hours	Discord + Miro	Darragh Quinn	Four domain stories; ubiquitous language review; backlog refinement

4.2 Participant Roles in Sessions

Team Member	Role in Sessions	Domain Expert Area
Caylum Hurley	Session 1 Facilitator; Context Map owner; notation enforcement	Book Club context; architecture style
Joe Considine	Domain expert proxy for Identity & Access; ACL contracts; policy writing	Authentication flows; role-based access

Oscar Zhou	Domain expert proxy for Catalogue; Hotspot H-04, H-05, H-11 owner	Book inventory; ISBN; soft-delete design
Daragh Downes	Session 2 Facilitator; Lending domain expert; H-01, H-02, H-06 owner	Loan lifecycle; overdue detection; DueDate VO
Vivian Jently	Session 3 Facilitator; Notification domain expert; backlog refinement	Event-driven notification policies; domain storytelling DS-03
Darragh Quinn	Book Club domain expert; WebSocket design (H-10); BACK-12 owner	Book Club context; real-time messaging; moderation

4.3 Miro Board Structure & Evidence



There features five sections on the Miro board, with a horizontal timeline at the top. Each section represents a single domain context. The board includes the following items:

- Domain Events timeline (orange stickies) - complete sequential flow of events, Sessions 1 and 2
- Commands (blue stickies) - one per event, aligned vertically with actor that triggers it
- Actors (yellow stickies) - Student, Staff, Admin, System, Notification Service
- Policy stickies (purple) - all 11 policies from Section B1.4 are posted above the event they apply to
- Hotspot stickies (red) - 11 hotspots marked, with resolution notes added in green during Sessions 2 and 3

- Aggregate boundaries (large pale-yellow frames) - UserAccount, Loan, Book, BookClub, Message

4.4 Decisions Log

Where all notable decisions made during sessions are recorded for traceability. Delta IDs and Backlog IDs are used to refer to decisions that changed the domain model.

Decision ID	Decision Made	Session	Rationale	Model / Backlog Impact
DEC-01	Use optimistic locking, not pessimistic locking, to resolve the last-copy race condition	Session 2	Pessimistic locking blocks concurrent read, slowing search performance. Optimistic locking with retry is good for borrowing patterns with few conflicts	BACK-01
DEC-02	Accept eventual consistency for catalogue search results; only check in real-time at borrow time	Session 2	Synchronous Catalogue calls on every query needed for real-time availability on every search result page. Slight UX staleness is fine; correctness is enforced at borrow.	H-02, BACK-02
DEC-03	Differentiate loan periods by Role: 14 days for Student, 28 days for staff	Session 2	Request from staff raised during session. Maps to real library practice. To keep the change local, made with DueDate VO factory method.	H-06, D-01, BACK-06
DEC-04	Use soft-delete for Books to keep integrity of loan history	Session 1	When hard deleting a book, loan records would be left with dangling foreign keys. Softdelete (deletedAt) removes book from active search but keeps historical records valid.	H-11, D-03, BACK-04
DEC-05	Notification v1 scope: in-app only; email integration to come in v2	Session 1	Email integration requires more SMTP setup. and more required for email integration. For an MVP, in-app notifications deliver the core requirement. Email added to backlog as low priority.	H-07, BACK-09

DEC-06	WebSocket (STOMP over SockJS) allows to chat with others in club in real time	Session 3	Latency would be introduced by polling in club discussions. Spring supports STOMP natively, which works with the Spring Boot backend. SockJS gives backup option for environments without WebSocket support.	H-10, D-05, BACK-12
DEC-07	Book Club invite-only mode: flag on BookClub entity; default is open-join	Session 3	The open-join default reduces friction for the MVP. Need for invite-only mode is a clear for users (discussed in DS-04 session). Set up as a boolean with new event path: MembershipRequestCreated.	H-09, D-05, D-07, BACK-11
DEC-08	AccountLockPolicy modelled as a Value Object, not a hardcoded constant	Session 2	Policies based on configuration are easier to test and maintain. The VO can change through configuration without changing the domain logic and is injected into AuthenticationService.	H-03, D-04, BACK-07

Part C: Implementation Architecture

1. Build / Run Instructions

The project can be run with docker-compose, it will initially take a long time to build but once its cached subsequent runs are much faster. Services can be ran individually as a springboot application, through intelliJ.

The frontend can be ran with npx. Using `npx serve .`` when inside of frontend/static

2. Services

There are 6 services:

- Gateway service;
- Catalogue Service;
- Lending Service;
- Identity Service;
- Notification Service;
- Book Club Service.

These are all springboot microservices. The gateway is responsible for routing calls to each one based on the path of the request.

The technology stack for each microservice is similar, using Spring Boot and PostgreSQL for persistent data store. The system can be run with Docker with the command `docker compose -up -build`.

For testing, individual services can be brought up and down with

`Docker compose start service-name`

And

`Docker compose stop service-name`

2.1 Gateway Service

- No domain layer
- JWTFilter enforces bounded context boundaries

2.2 Catalogue Service

- Book and BookCopy are separate aggregate roots
- ISBN and AvailabilityStatus are value objects
- InventoryService enforces the availability invariant
- `application/acl/` translates from lending

2.3 Lending Service

- Loan is the aggregate root
- DueDate is a value object
- LoanIssuanceService and OverdueDetectionService are domain services
- CatalogueACL prevents model leakage

2.4 Identity Service

- User is the aggregate root
- Role and LockoutPolicy are value objects
- LoginUserUseCase and RegisterUserUseCases handle command logic

2.5 Notification Service

- Subscriber-only context
- Notification is the aggregate root

- LendingEventTranslator and ClubEventTranslator act as ACLs to decouple from upstream contracts
- Deduplication enforced by NotificationRepository

2.6 Book Club Service

- BookClub and Message are separate aggregate roots (to avoid contention)
- ClubRole value object enforces the single owner invariant
- BookClubACL maps Identity responses to an internal ClubMemberProfileVO

3 Configuration Tools

Configuration was mostly hard coded into the project. Through the standard application.properties files used in Springboot.

4. Environment Separation

Used docker compose for building the microservices together in a consistent way. For a production release the same compose file could be used with a few modifications, namely changing default passwords and switching to nginx, https for a proper frontend, etc.

5. Versioning

Git was used for versioning and keeping track of the code. The repository is hosted on Github.

6. Patterns

Circuit breakers were implemented for the catalogue client using the resilience4j package.

The gateway service implements spring cloud gateway filters on each route. It specifically uses a 10-request sliding window, 50% failure threshold, 10s open state, 3 calls in half open configuration. The status of which can be seen at <http://localhost:8761/> through Spring Eureka, only when the project is running of course.

The actuator endpoints are exposed on each service's own port

- Gateway: localhost:9920/actuator/circuitbreakers
- Lending: localhost:9923/actuator/circuitbreakers

The script test-resilience uses these and displays output, they can be queried with a browser/postman as well.

TimeLimiter is configured for timeouts at 5s for all five circuit breaker instances.

Retries are used with the @Retry annotation imported from the resilience4j package.

For fallbacks the lending service throws its own custom exception indicating the service is unavailable.

The lending service is the only place that makes an async call to another service, being the Catalogue service. This is why it's the only place that has retries + a fallback exception.

7. Integration Tests

Each service has integration tests that can be run with Junit. They use test containers to mock the database and other external services. It is easiest to run these inside of IntelliJ which can get the required dependencies.

8. Traces & Logs

There are two scripts in the root directory of the project that can be used to test the containers. These have logs of the output to verify if things worked or not. Logs can be checked through docker with ``docker compose logs elib-lending``, replacing elib-lending with the microservice name.

9. Screenshots

```
[INFO] Sending more requests to trigger circuit breaker OPEN state...
[INFO] Request 1 → HTTP 503
[INFO] Request 2 → HTTP 503
[INFO] Request 3 → HTTP 503
[INFO] Request 4 → HTTP 503
[INFO] Request 5 → HTTP 503
```

— PHASE 3 – Observe Circuit Breaker State —

```
[INFO] Checking circuit breaker state via actuator...
```

```
GET /actuator/circuitbreakers
```

```
{
  "circuitBreakers": {
    "catalogueService": {
      "bufferedCalls": 10,
      "failedCalls": 8,
      "failureRate": "80.0%",
      "failureRateThreshold": "50.0%",
      "notPermittedCalls": 10,
      "slowCallRate": "0.0%",
      "slowCallRateThreshold": "100.0%",
      "slowCalls": 0,
      "slowFailedCalls": 0,
      "state": "OPEN"
    }
  }
}
```

Test scripts demonstrating the circuit breaker state, test-resilience.sh

http://localhost:8761

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-04-17 11:00:09
Data center	default	Uptime	00:09
		Lease expiration enabled	true
		Renews threshold	11
		Renews (last min)	24

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ELIB-BOOKCLUB	n/a (1)	(1)	UP (1) - 7a27c368d386.elib-bookclub.9929
ELIB-CATALOGUE	n/a (1)	(1)	DOWN (1) - 5e8e52ad5ce5.elib-catalogue.9922
ELIB-GATEWAY	n/a (1)	(1)	UP (1) - b4c021ee4d9c.elib-gateway.9920
ELIB-IDENTITY	n/a (1)	(1)	UP (1) - b8bf2d12e7f0.elib-identity.9921
ELIB-LENDING	n/a (1)	(1)	UP (1) - 0df550c17221.elib-lending.9923
ELIB-NOTIFICATION	n/a (1)	(1)	UP (1) - 5c8bbb292bbb.elib-notification.9924

General Info

Name	Value
total-avail-memory	160mb

Spring Eureka showing services if they are down.

My Loans

Books you have borrowed. Return them before the due date.

Active 5

Test Book

Borrowed: 17 Apr 2026 • Due: 1 May 2026

14d left

ACTIVE

Return

Clean Code2

Borrowed: 17 Apr 2026 • Due: 1 May 2026

14d left

ACTIVE

Return

Test Book

Borrowed: 17 Apr 2026 • Due: 1 May 2026

14d left

ACTIVE

Return

Clean Code2

Borrowed: 17 Apr 2026 • Due: 1 May 2026

14d left

ACTIVE

Return

Test Book

Borrowed: 17 Apr 2026 • Due: 1 May 2026

14d left

ACTIVE

Return

History 3

Clean Code2

Borrowed: 15 Apr 2026 • Due: 29 Apr 2026 • Returned: 15 Apr 2026

RETURNED

Clean Code2

Borrowed: 15 Apr 2026 • Due: 29 Apr 2026 • Returned: 15 Apr 2026

RETURNED

Clean Code2

Borrowed: 15 Apr 2026 • Due: 29 Apr 2026 • Returned: 15 Apr 2026

RETURNED

Frontend loans

[Books](#)[My Loans](#)[Book Clubs](#)

Browse Books

Search the catalogue and borrow books.

[+ Add Book](#)

Clean Code2

by Robert Martin • 2008

Programming

4/6 available

Test Book

by abc

2/5 left

Frontend Books

← Back to Clubs

test

test

 **Messages**

 **Members**

You

15 Apr

aaa

[Edit](#) [Delete](#)

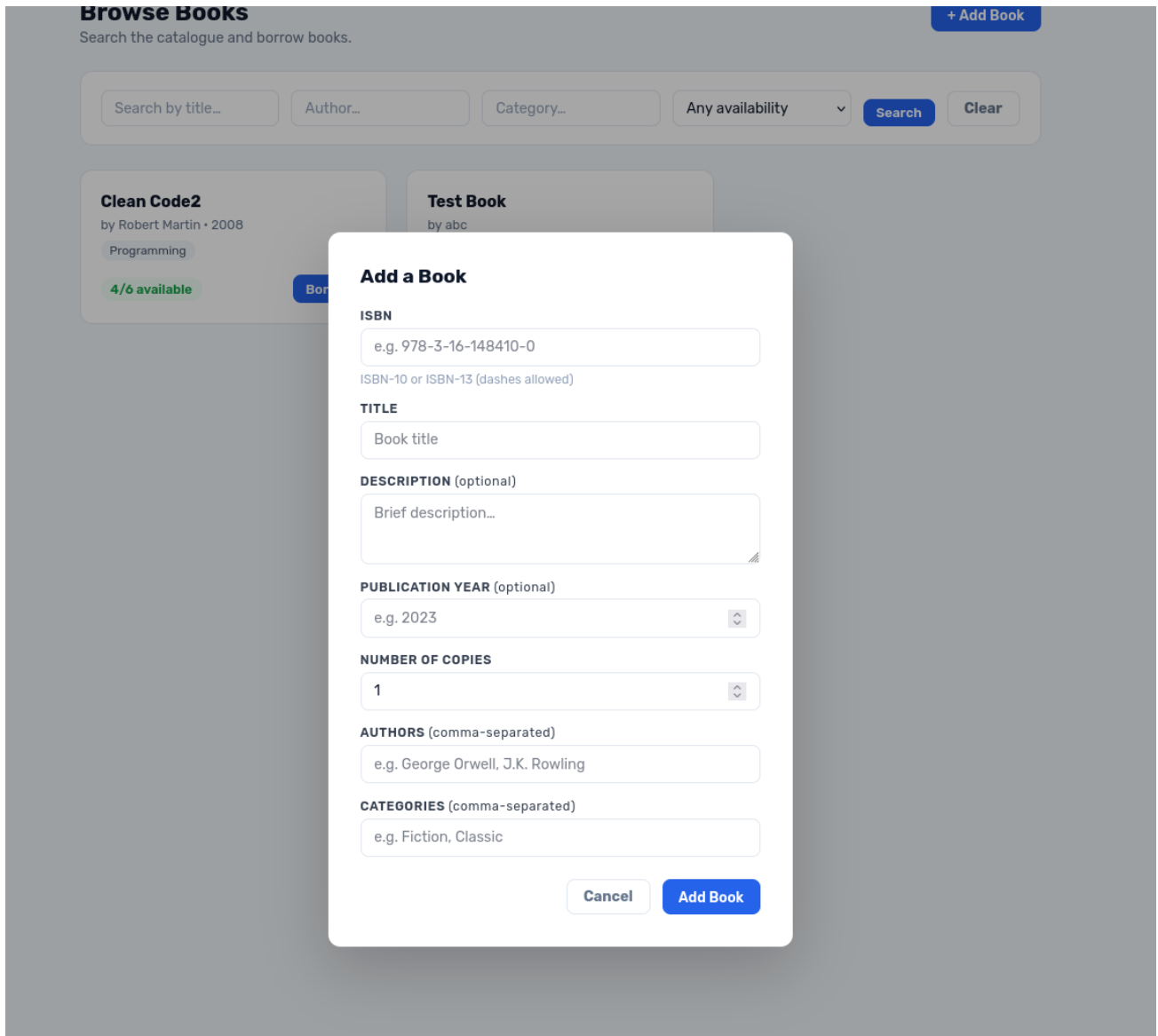
You

1h ago

i love harry potter

[Edit](#) [Delete](#)

Frontend book clubs



Can add book through UI